

From product brief to working system

The architecture, contracts and operating details behind a useful first release.

One worked operations platform. Follow the product slice, choose the boundaries, specify failure behaviour and leave a release someone else can maintain.

6 PAGES / DIAGRAMS + CONTRACTS + RELEASE CHECKS

PRODUCT ENGINEERING / BACKEND SYSTEMS / INTEGRATIONS

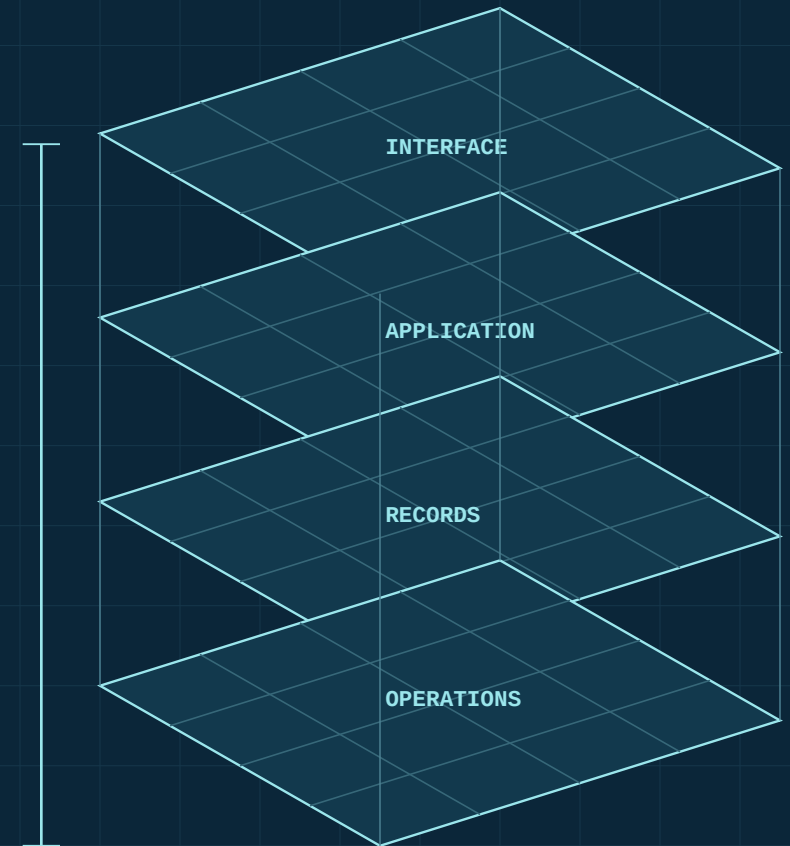


PLATE 01 / A SYSTEM IN LAYERS

Original design study

Start with the work someone is waiting to do.

Worked design: a customer-operations workspace for a small software team. These are explicit project assumptions, not claims about a client deployment.

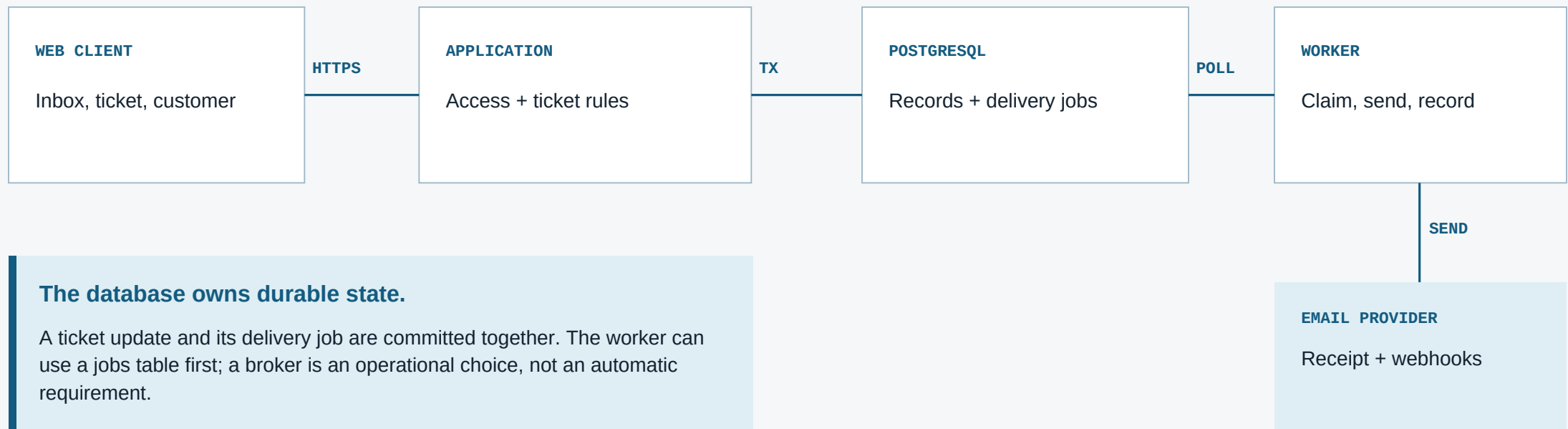
Decision	Example brief	Acceptance evidence
Who uses it?	Support agents work across assigned customer accounts. A workspace admin manages membership.	An agent can open an assigned ticket and see its history; another workspace remains inaccessible.
What is the first slice?	Receive a request, assign an owner, add a note and send a reply. Keep the customer record alongside the conversation.	One complete ticket moves from incoming to resolved with an attributable history.
What is deliberately later?	AI drafting, advanced dashboards, workflow builders and custom permission editors.	The first release remains useful when all four are absent.
What changes the design?	Assume up to 100 workspaces and 30 concurrent staff sessions for planning. Measure actual use before scaling decisions.	Agree representative dataset size and traffic shape before a load test; assumptions are not benchmark results.

Write the awkward case into the brief: two agents open the same ticket, an agent loses access, or a reply request times out. The interface must explain what happened and leave the work recoverable.

02 / CHOOSE BOUNDARIES

Keep the first architecture small enough to explain.

A modular application, a relational database and a background worker are sufficient for this design. A separate service needs a concrete reason: ownership, isolation or independent scaling.



Core records: **workspace, membership, customer, ticket, message, delivery job**. Keep workspace identity on records and enforce relationships in the database as well as application rules. [1]

The browser owns the visible next step.

Show queued, sent, failed or unconfirmed. A spinner is temporary feedback; it cannot be the only record of a customer action.

An endpoint name leaves most of the job unfinished.

Define the caller, state change and failure response together. The same rule must hold whether a request comes from the interface, an integration or a background task.

Contract	Expected behaviour	Case to prove
GET /tickets?cursor=...	Return tickets within the caller's workspace and allowed scope. Use a stable created_at + id order; document filtering and cursor lifetime.	Two tickets share a timestamp. Paging produces a stable next cursor without using an unstable offset.
POST /tickets/{id}/messages	Validate workspace membership, ticket access, recipient and payload. Commit a queued message and job; return 202 with message_id and a status URL.	A cross-workspace ticket ID reveals neither the ticket nor its content. Queued is visible before the provider finishes.
POST /tickets/{id}/assign	Update only if the submitted record version is current. Return a documented conflict response when the version has changed.	Two agents assign different owners. One update wins; the other sees the current state and can choose again.

Authorisation is a server decision.

Check the caller against the resource on every request, including downloads and job-status lookups. An obscure ID or hidden button is not an access policy. [2]

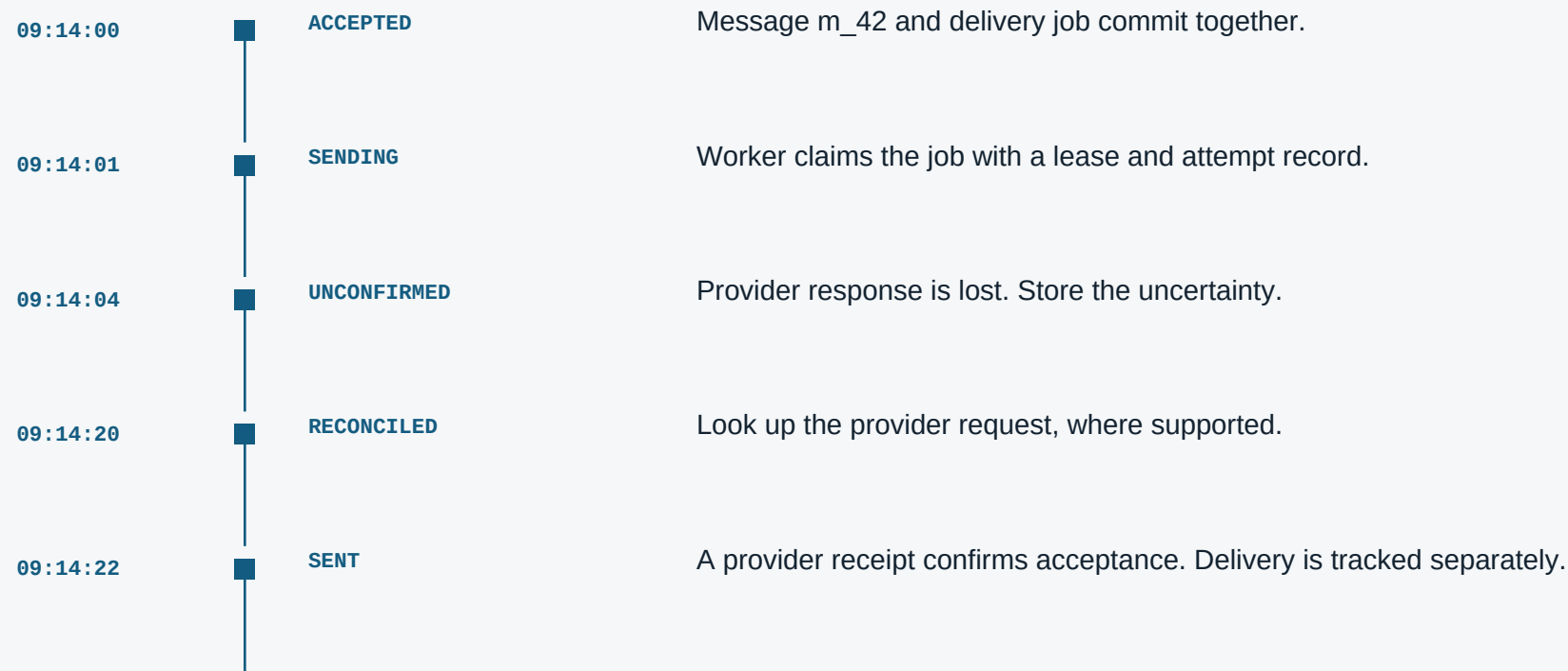
A retry needs an identity.

For message creation, bind an idempotency key to workspace, operation and payload fingerprint. Reuse returns the existing message; a different payload with the same key is a conflict.

04 / HANDLE INCOMPLETE INFORMATION

A timeout should not erase the history.

The example below follows one reply. Application state, a provider receipt and recipient delivery are different observations; keep them separate.



When reconciliation is unavailable

Leave the attempt unconfirmed and route it for review. This design cannot guarantee exactly-once delivery across an external provider without the necessary provider support.

Recover the worker, too

Test a crash after provider acceptance but before the receipt is stored. Expiring a worker lease must not blindly cause another send. Bound retries and back off transient failures. [3]

Make the handover useful on a bad afternoon.

The next engineer should be able to trace an action, understand its current state and make a controlled change without reconstructing the entire project.

Review	Evidence to keep	Worked release check
Product path	A short recording plus the record IDs from create, assign, reply and resolve.	An agent can complete the path on a narrow screen, including empty, loading, error and conflict states.
Access and data	Cross-workspace tests, revoked membership checks, schema migration notes and a backup-restore result.	A session that loses access cannot read a ticket or its attachments through an old link.
Operations	One trace connecting request, database work and background processing; alert and runbook ownership. [4]	An unconfirmed send has a searchable message ID, an owner and a documented reconciliation path.
Rollout	Named release owner, representative test results, previous version and a migration recovery plan.	Enable for a small internal workspace first. A feature switch stops new sends while leaving receipts available.

Before the first build session: agree the thin product slice, access rules, system of record and acceptance evidence. Record each unresolved dependency with an owner; revisit architecture when the evidence changes.

PRIMARY REFERENCES / LINKED

- [1 PostgreSQL constraints](#)
- [2 OWASP authorisation guidance](#)
- [3 AWS timeouts and retries](#)
- [4 OpenTelemetry traces](#)